3. Files, Links, and Metadata

03. Files, Links, and Metadata

Phase: Phase 1 — Shell and filesystem
Estimated time: 1–2 days
Slug: `03-files-meta`

Learning objectives

- Inspect files with file, stat, du, df
- Distinguish hard links vs symbolic links
- Understand inodes at a practical level
- Use touch, install, and install-like patterns

Prerequisites

Complete prior numbered lessons. Prefer a real Linux VM/container for Linux-specific topics. Type every command yourself — muscle memory beats skimming.

Why this matters

Everyday work is creating, copying, moving and linking files. Underneath, Linux files are identified by inodes, and names are only entries in directories pointing to them. Understanding this explains why a hard link survives deleting the original, why a symlink can dangle, and why `mv` within one filesystem is instant while across filesystems it copies.

Core concepts

An *inode* stores metadata (type, owner, permissions, size, timestamps, link count, data location) but not the file name. A directory is a table of name-to-inode entries. A *hard link* is another name for the same inode; the data is removed only when the link count reaches 0 and no process holds it open. A *symbolic (soft) link* is a small separate file containing a path; it can point to a directory, across filesystems, or to something that no longer exists.

```
hard links                          symbolic link
a.txt --\                           link.txt --> inode 77 (type l)
         +--> inode 42 --> data               data: "a.txt"  (a path string)
b.txt --/    (link count 2)                        |
                                                   +--> looks up a.txt by name
```

Property	Hard link	Symbolic link
Own inode	No, shares target's	Yes
Cross filesystem	No	Yes
Link to directory	No (normally)	Yes
Survives target deletion	Yes, data stays	No, becomes dangling
`ls -l` first char	`-`	`l`
Create	`ln a b`	`ln -s a b`

Metadata commands: `ls -l` (type, perms, links, owner, size, mtime), `stat` (all fields including inode), `file` (guess content type). File types by the first character of `ls -l`: `-` regular, `d` directory, `l` symlink, `c`/`b` devices, `s` socket, `p` pipe.

Timestamps: mtime (content changed), ctime (inode metadata changed), atime (read). `touch` updates them or creates an empty file.

Worked examples

Hard versus soft:

```
cd /tmp && echo hi > a.txt
ln a.txt hard.txt
ln -s a.txt soft.txt
ls -li a.txt hard.txt soft.txt
# sample output: 131090 -rw-r--r-- 2 student student 3 Oct 21 10:00 a.txt
# sample output: 131090 -rw-r--r-- 2 student student 3 Oct 21 10:00 hard.txt
# sample output: 131091 lrwxrwxrwx 1 student student 5 Oct 21 10:00 soft.txt ->
a.txt
```

Same inode and link count 2 for the hard pair; the symlink has its own inode and shows its target.

Delete the original:

```
rm a.txt
cat hard.txt
# sample output: hi
cat soft.txt
# sample output: cat: soft.txt: No such file or directory
```

The hard link still holds the data; the soft link dangles.

Inspect metadata:

```
stat -c '%n %i %h %F' hard.txt soft.txt
# sample output: hard.txt 131090 1 regular file
# sample output: soft.txt 131091 1 symbolic link
```

```
file soft.txt
# sample output: soft.txt: broken symbolic link to a.txt
```

Test the type in a script:

```
[ -L soft.txt ] && echo symlink
# sample output: symlink
readlink soft.txt
# sample output: a.txt
```

Mini-lab

1. `mkdir -p /tmp/lab3 && cd /tmp/lab3 && echo data > one`.
2. `ln one two` and `ln -s one three`.
3. Run `ls -li` and confirm `one` and `two` share an inode number.
4. Append with `echo more >> two` and `cat one`; both names show the same content.
5. Run `stat one` and read Links, Inode, Modify and Change times.
6. `rm one`, then `cat two` (works) and `cat three` (fails).
7. Recreate `one` with `echo new > one` and `cat three`; the symlink now works again because it resolves by name.
8. Try `ln /etc/hostname hostlink` on a different filesystem (for example from /tmp tmpfs) and read the error “Invalid cross-device link”.

Decision guide / failure modes

Use a symlink for shortcuts, version switches (`current -> release-42`), links across filesystems, and links to directories. Use a hard link for extra names of the same file on one filesystem, as backup tools do to save space. When unsure, choose a symlink; it is visible and easy to inspect.

Common failures:

- Dangling symlinks after moving the target; relative link targets are resolved from the link’s directory, not from your current directory.
- Expecting a hard link to follow an editor that replaces the file (many editors write a new file, which breaks the shared inode).
- `ln -s target link` argument order confusion: target first, link name second.
- `rm link/` with a trailing slash on a symlink to a directory may act on the target contents.
- `cp` follows or copies links differently depending on options; use `cp -a` to preserve links and metadata.

Common mistakes

- Assuming `rm` of a symlink deletes the target (it removes the link)
- Hard-linking directories (usually forbidden)
- Ignoring disk full (`df`) when services fail mysteriously

Exercises

Work in `exercises/03-files-meta/`. Read that folder's `README.md` fully before coding.

```
cd exercises/03-files-meta
chmod +x test.sh exercise.sh
./test.sh
```

What you will build

Implement link_kind(ls_first_char_or_name). The input is a string describing how a link was made: either the word "soft" or "hard" (case-insensitive, surrounding whitespace ignored), or the command used: "ln -s" means soft and plain "ln" means hard. Return the lowercase string "soft" for symbolic links and "hard" for hard links. For any unrecognized input return "unknown". The point is to connect the commands to the concepts: ln -s creates a symbolic (soft) link with its own inode, plain ln creates another name for the same inode.

Study loop

- **A (Drill):** make `./test.sh` pass
- **B (Apply):** run 2–3 lab commands from this lesson on a real Linux host; paste notable output into your notes
- **C (Challenge):** rewrite the exercise from memory the next day

Do not open `solutions/03-files-meta/` until you have an honest failing attempt.

Self-check

Try to answer without scrolling up:

1. Hard link vs symlink?
2. What does `stat` show that `ls -l` might not?
3. What does `df -h` report?

Further reading

- https://man7.org/linux/man-pages/man1/ln.1.html